Compte rendu de travaux pratiques

Programmation pour cible embarquée

Lecture de boutons poussoirs et commande d'une LED :
débogage des registres GPIO sous Keil µVision

Enseignant de TP

M. Atef Ftirich

Réalisé par

Bouabdallah Mohamed Yacine

Ben Hmida Ahmed

Arrami Mohamed Ali

Classe : LISI3

Groupe : A

Année universitaire : 2026/2027

Sommaire

1. Introduction et objectifs..... 2
2. Matériel et schéma de câblage..... 2
2.1 Matériel utilisé..... 2
2.2 Schéma du montage..... 2
2.3 Principe de la résistance de rappel (pull-down)..... 4
2.4 Affectation des broches..... 4
3. Rappel : les registres GPIO utilisés..... 4
4. Méthode de débogage sous Keil µVision..... 5
5. Code 1 : un bouton poussoir commande la LED..... 5
5.1 Programme..... 6
5.2 Explication du programme..... 6
5.3 Résultat sur la carte..... 7
5.4 Débogage et registres..... 7
6. Code 2 : bouton vert pour allumer, bouton rouge pour éteindre..... 8
6.1 Programme..... 9
6.2 Explication du programme..... 9
6.3 Résultat sur la carte..... 10
6.4 Débogage et registres..... 10
7. Remarque : BSRR et ODR en simulation..... 11
8. Conclusion..... 11
Remerciements..... 12

1. Introduction et objectifs

Après avoir commandé des sorties du microcontrôleur STM32F407 lors des séances précédentes, ce TP porte sur la lecture d'entrées numériques. Des boutons poussoirs externes, montés sur une breadboard avec une résistance de rappel, sont reliés à la carte STM32F4 Discovery et commandent une LED. Les programmes sont écrits en C sous Keil µVision en accédant directement aux registres, sans générateur de code.

Objectifs du TP :

- réaliser le montage d'un bouton poussoir avec une résistance de rappel vers la masse (pull-down) ;
- configurer une broche en entrée (MODER, PUPDR) et lire son état dans le registre IDR ;
- commander une LED avec un bouton poussoir (code 1), puis avec deux boutons, l'un pour allumer et l'autre pour éteindre (code 2) ;
- utiliser le débogueur de Keil pour observer la valeur des registres GPIO à chaque étape et vérifier le fonctionnement des programmes.

2. Matériel et schéma de câblage

2.1 Matériel utilisé

Composant	Rôle
Carte STM32F4 Discovery (STM32F407VG)	Microcontrôleur Cortex-M4 programmé en C, avec programmateur ST-LINK intégré.
Câble USB (mini-USB)	Alimentation de la carte et téléversement du programme.
Breadboard et fils de liaison	Réalisation des connexions sans soudure.
Deux boutons poussoirs (vert et rouge)	Entrées de commande : un seul bouton pour le code 1, les deux pour le code 2.
Résistances de 1 kΩ	Résistances de rappel vers GND (pull-down) des boutons.
Une LED + résistance de 220 Ω	Sortie commandée ; la résistance limite le courant dans la LED.
Keil µVision 5, pack STM32F4xx_DFP 2.17.1	Écriture, compilation, débogage et téléversement (CMSIS CORE + Device Startup).

2.2 Schéma du montage

Le schéma de principe a été établi au tableau pendant la séance : le bouton est placé entre l'alimentation et la broche d'entrée, une résistance de 1 kΩ relie cette broche à la masse, et la LED est reliée à une sortie à travers une résistance de 220 Ω.

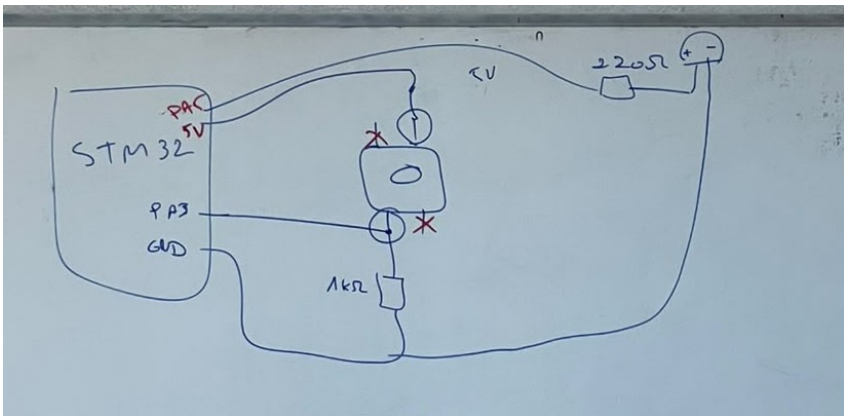


Figure 1 – *Schéma de principe réalisé au tableau pendant la séance*

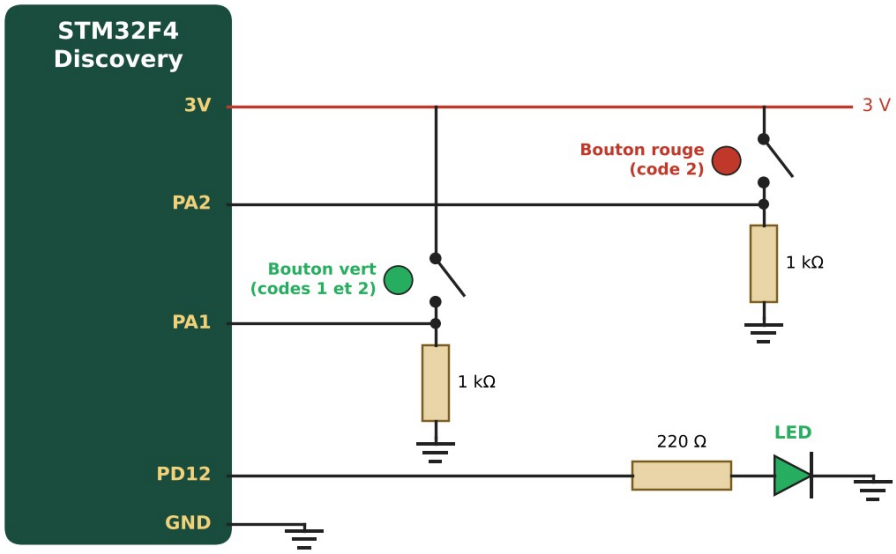


Figure 2 – *Schéma de câblage du montage complet (codes 1 et 2)*

La photo annotée ci-dessous fait le lien entre le schéma et le montage réel : chaque composant est repéré avec la broche du microcontrôleur à laquelle il est relié.

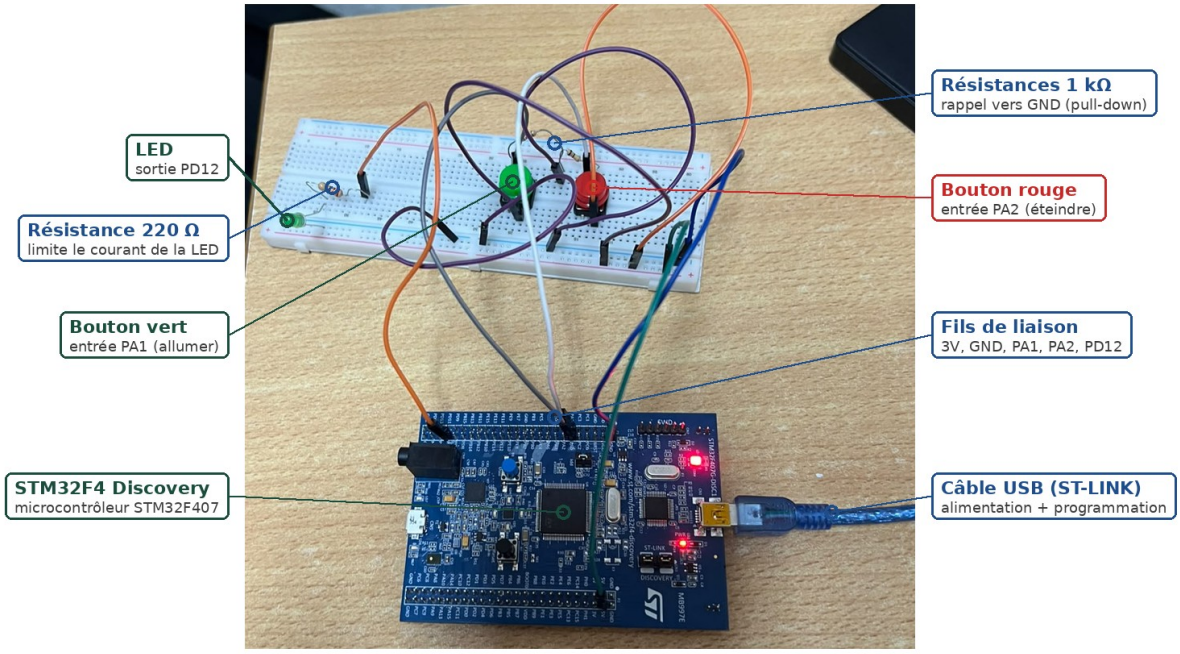


Figure 3 – *Montage réel annoté (code 2) : boutons, résistances, LED et carte Discovery*

2.3 Principe de la résistance de rappel (pull-down)

Une entrée laissée « en l'air » n'a pas de niveau défini : elle capte les parasites et peut être lue tantôt à 0, tantôt à 1. La résistance de rappel fixe un niveau stable lorsque le bouton est relâché :

- **bouton relâché :** la broche est reliée à la masse par la résistance de 1 kΩ, elle lit 0 ;
- **bouton appuyé :** la broche est reliée directement au 3 V, elle lit 1. La résistance limite alors le courant entre 3 V et GND à environ 3 mA.

En plus de la résistance externe, nous avons activé la résistance de pull-down interne du microcontrôleur (code `10` dans le registre PUPDR). Les deux résistances vont dans le même sens et la logique reste **appuyé = 1, relâché = 0**.

2.4 Affectation des broches

Broche	Mode	Rôle	Code
PA1	Entrée, pull-down	Bouton vert (le seul bouton du code 1)	1 et 2
PA2	Entrée, pull-down	Bouton rouge	2
PD12	Sortie	LED externe (aussi reliée à la LED verte LD4 de la carte)	1 et 2
3V / GND	Alimentation	Alimentation des boutons et masse commune	1 et 2

Le choix de PA1, PA2 et PD12 permet d'éviter PA0, déjà reliée au bouton utilisateur bleu de la carte Discovery. PD12 a l'avantage d'allumer aussi la LED verte LD4 présente sur la carte, ce qui donne un second témoin visuel.

3. Rappel : les registres GPIO utilisés

Registre	Adresse (port)	Rôle
`RCC->AHB1ENR`	0x40023830	Active l'horloge des ports : bit 0 pour GPIOA, bit 3 pour GPIOD (valeur attendue 0x9).

Registre	Adresse (port)	Rôle
`MODER`	+0x00	Mode de chaque broche sur 2 bits : 00 entrée, 01 sortie.
`PUPDR`	+0x0C	Résistance interne sur 2 bits : 00 aucune, 01 pull-up, 10 pull-down.
`IDR`	+0x10	Lecture de l'état des broches en entrée (bit n = état de la broche n).
`ODR`	+0x14	État des broches en sortie.
`BSRR`	+0x18	Bits 0 à 15 : mise à 1 de la broche ; bits 16 à 31 : mise à 0.

Les ports utilisés commencent aux adresses `0x40020000` (GPIOA) et `0x40020C00` (GPIOD). Le registre `BSRR` modifie une seule broche en une seule écriture, sans toucher aux autres : écrire `1U << 12` met PD12 à 1, écrire `1U << 28` (12 + 16) la met à 0.

4. Méthode de débogage sous Keil µVision

Pour observer les registres, chaque programme a été exécuté dans le débogueur de Keil (menu Debug > Start/Stop Debug Session). Comme la carte n'était pas reliée au PC au moment des captures, nous avons utilisé le simulateur intégré de Keil. Un petit fichier d'initialisation, sim.ini, est déclaré dans les options du débogueur pour rendre accessible la zone mémoire des périphériques :

Fichier sim.ini

```
MAP 0x40000000, 0x5FFFFFFF READ WRITE   /* rend la zone des périphériques accessible */
```

La démarche suivie pour chaque code est la suivante :

- placer un point d'arrêt dans la boucle while (1), lancer l'exécution (F5) jusqu'à ce point ;
- afficher les registres dans la fenêtre **Watch 1**, par exemple `((GPIO_TypeDef *)0x40020000)->IDR` pour le registre IDR du port A ;
- simuler l'appui sur un bouton en écrivant dans IDR depuis la fenêtre Command, par exemple `_WDWORD(0x40020010, 0x2)` pour mettre PA1 à 1 ;
- exécuter une itération de la boucle et relever la valeur écrite dans BSRR.

5. Code 1 : un bouton poussoir commande la LED

Dans ce premier programme, un seul bouton (le vert, relié à PA1) est utilisé : la LED est allumée tant que le bouton est maintenu appuyé, et s'éteint dès qu'il est relâché.

5.1 Programme

```
/* Code 1 : un bouton poussoir commande une LED
   Bouton : PA1 (entrée, bouton relié à 3V, résistance de rappel vers GND)
   LED    : PD12 (sortie, LED externe + LED verte LD4 de la carte) */
#include "stm32f407xx.h"

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIODEN;

    GPIOD->MODER &= ~(3U << (12 * 2));     /* PD12 en sortie (01) */
    GPIOD->MODER |=  (1U << (12 * 2));

    GPIOA->MODER &= ~(3U << (1 * 2));      /* PA1 en entrée (00) */
    GPIOA->PUPDR &= ~(3U << (1 * 2));      /* PA1 : pull-down (10) */
    GPIOA->PUPDR |=  (2U << (1 * 2));

    while (1)
    {
        if (GPIOA->IDR & (1U << 1))          /* bouton appuyé : PA1 = 1 */
        {
            GPIOD->BSRR = (1U << 12);        /* PD12 = 1 : LED allumée */
        }
        else                                 /* bouton relâché : PA1 = 0 */
        {
            GPIOD->BSRR = (1U << (12 + 16)); /* PD12 = 0 : LED éteinte */
        }
    }
}
```

5.2 Explication du programme

Instruction	Rôle
`RCC->AHB1ENR |= ...GPIOAEN | ...GPIODEN`	Active l'horloge des ports A (bouton) et D (LED). Sans cette ligne, les registres des ports ne réagissent pas.
`GPIOD->MODER ... (12 * 2)`	Efface puis écrit 01 dans les bits 24-25 : PD12 en sortie.
`GPIOA->MODER &= ~(3U << (1 * 2))`	Met les bits 2-3 à 00 : PA1 en entrée.
`GPIOA->PUPDR |= (2U << (1 * 2))`	Écrit 10 dans les bits 2-3 : résistance de pull-down interne sur PA1.
`if (GPIOA->IDR & (1U << 1))`	Isole le bit 1 de IDR : la condition est vraie lorsque PA1 = 1, c'est-à-dire bouton appuyé.
`GPIOD->BSRR = (1U << 12)`	Met PD12 à 1 : la LED s'allume.
`GPIOD->BSRR = (1U << (12 + 16))`	Met PD12 à 0 : la LED s'éteint.

La boucle while (1) lit le bouton en permanence (scrutation, ou polling) : l'état de la LED suit donc immédiatement celui du bouton.

5.3 Résultat sur la carte

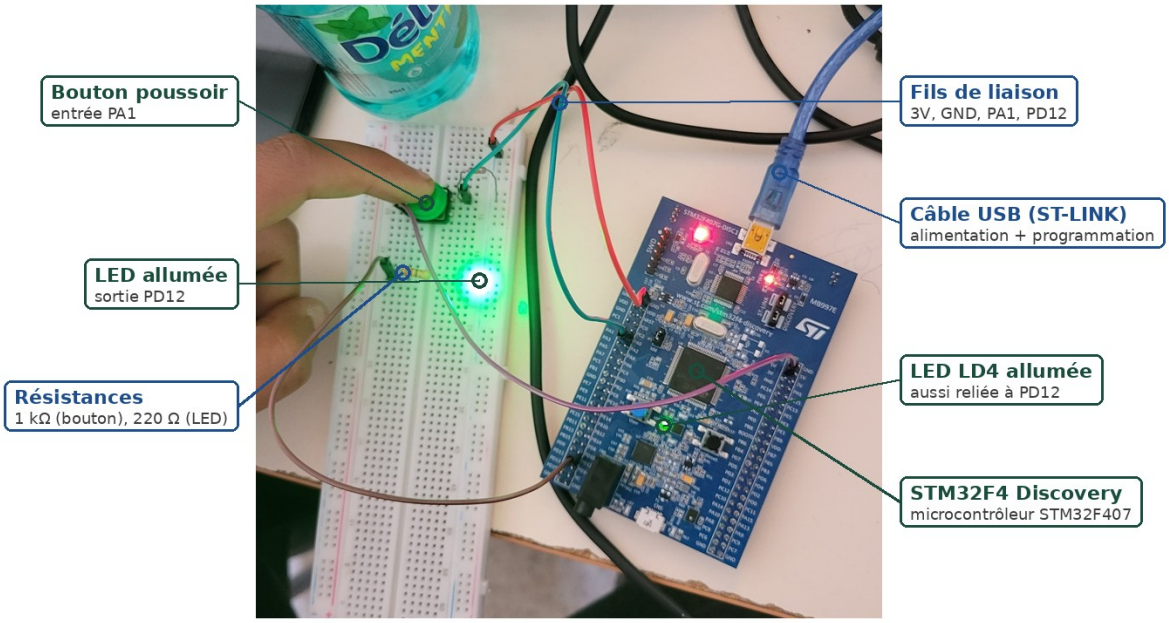


Figure 4 – *Montage réel annoté du code 1 : un seul bouton poussoir commande la LED*

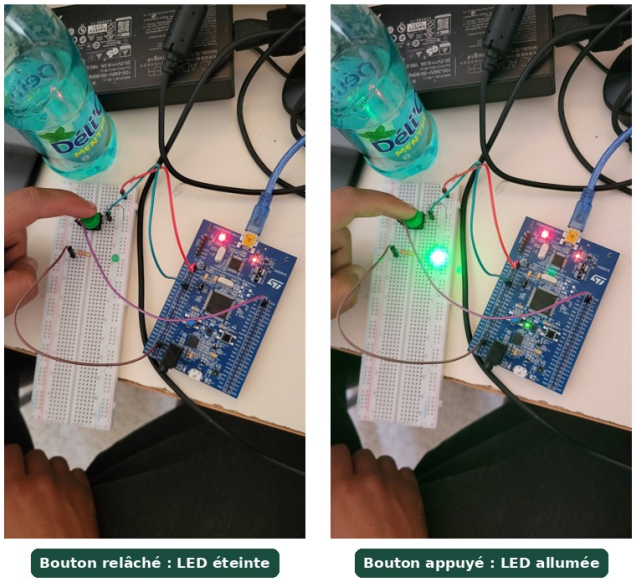


Figure 5 – *Code 1 sur la carte : LED éteinte au repos, allumée pendant l'appui*

Lorsque le bouton est appuyé, la LED externe et la LED verte LD4 de la carte s'allument ; elles s'éteignent dès que le bouton est relâché.

5.4 Débogage et registres

Deux situations ont été observées dans le débogueur : bouton relâché (IDR = 0), puis bouton appuyé (IDR = 0x2, simulé avec _WDWORD).

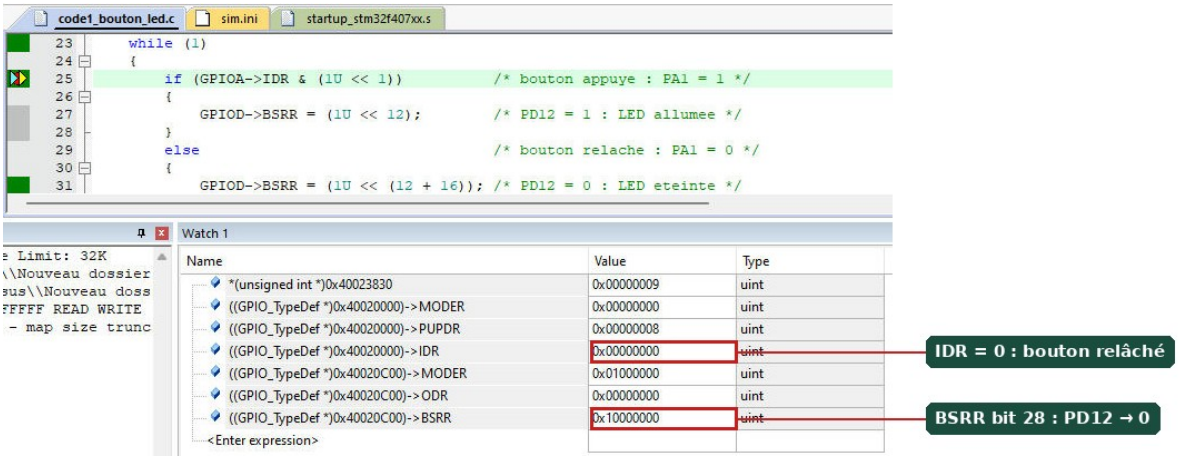


Figure 6 – *Code 1, bouton relâché : IDR = 0, le programme écrit le bit 28 de BSRR*

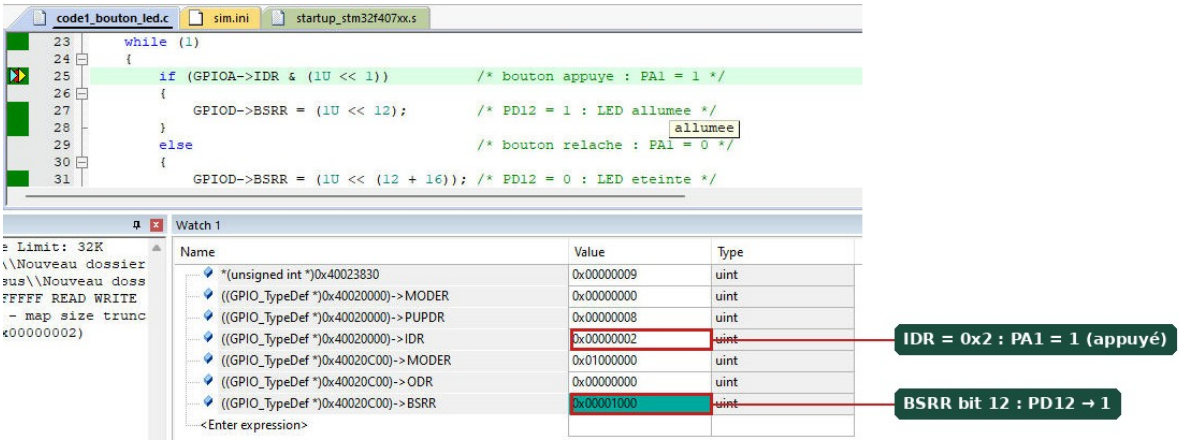


Figure 7 – *Code 1, bouton appuyé : IDR = 0x2, le programme écrit le bit 12 de BSRR*

Registre	Bouton relâché	Bouton appuyé	Interprétation
`RCC->AHB1ENR`	0x00000009	0x00000009	Bits 0 et 3 : horloges de GPIOA et GPIOD actives.
`GPIOA->MODER`	0x00000000	0x00000000	PA1 en entrée (00).
`GPIOA->PUPDR`	0x00000008	0x00000008	Bits 2-3 = 10 : pull-down sur PA1.
`GPIOA->IDR`	0x00000000	0x00000002	Bit 1 = état du bouton sur PA1.
`GPIOD->MODER`	0x01000000	0x01000000	Bits 24-25 = 01 : PD12 en sortie.
`GPIOD->BSRR`	0x10000000	0x00001000	Bit 28 : PD12 → 0 ; bit 12 : PD12 → 1.

Les valeurs relevées confirment le fonctionnement attendu : la configuration est correcte (horloges, modes, pull-down) et l'écriture dans BSRR change selon l'état du bouton lu dans IDR.

6. Code 2 : bouton vert pour allumer, bouton rouge pour éteindre

Le second programme utilise les deux boutons : un appui sur le bouton vert (PA1) allume la LED, un appui sur le bouton rouge (PA2) l'éteint. Contrairement au code 1, la LED garde son état quand on relâche les boutons : elle se comporte comme une mémoire (fonction marche/arrêt).

6.1 Programme

```
/* Code 2 : deux boutons poussoirs commandent une LED
   Bouton vert  : PA1 -> allume la LED
   Bouton rouge : PA2 -> éteint la LED
   LED          : PD12 */
#include "stm32f407xx.h"

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIODEN;

    GPIOD->MODER &= ~(3U << (12 * 2));     /* PD12 en sortie */
    GPIOD->MODER |=  (1U << (12 * 2));

    /* PA1 et PA2 en entrée avec pull-down */
    GPIOA->MODER &= ~((3U << (1 * 2)) | (3U << (2 * 2)));
    GPIOA->PUPDR &= ~((3U << (1 * 2)) | (3U << (2 * 2)));
    GPIOA->PUPDR |=  ((2U << (1 * 2)) | (2U << (2 * 2)));

    while (1)
    {
        if (GPIOA->IDR & (1U << 1))          /* bouton vert appuyé */
        {
            GPIOD->BSRR = (1U << 12);        /* LED allumée */
        }
        if (GPIOA->IDR & (1U << 2))          /* bouton rouge appuyé */
        {
            GPIOD->BSRR = (1U << (12 + 16)); /* LED éteinte */
        }
    }
}
```

6.2 Explication du programme

Instruction	Rôle
`GPIOA->MODER &= ~((3U << 2) | (3U << 4))`	Met PA1 (bits 2-3) et PA2 (bits 4-5) en entrée en une seule instruction.
`GPIOA->PUPDR |= ((2U << 2) | (2U << 4))`	Pull-down sur PA1 et PA2 : PUPDR = 0x28 (binaire 10 1000).
`if (GPIOA->IDR & (1U << 1))`	Bouton vert appuyé : écriture du bit 12 de BSRR, la LED s'allume.
`if (GPIOA->IDR & (1U << 2))`	Bouton rouge appuyé : écriture du bit 28 de BSRR, la LED s'éteint.
Aucun else	Si aucun bouton n'est appuyé, rien n'est écrit : la LED conserve son dernier état.

Si les deux boutons sont appuyés en même temps, le second test est exécuté en dernier dans chaque tour de boucle : c'est donc le bouton rouge qui l'emporte et la LED reste éteinte. Ce comportement est un choix de sécurité classique : l'arrêt est prioritaire sur la marche.

6.3 Résultat sur la carte

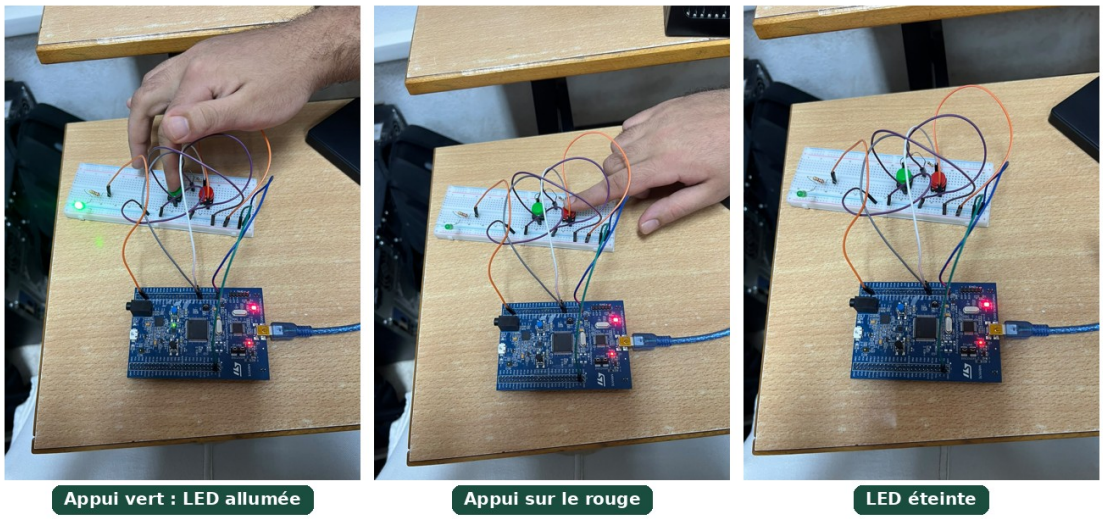


Figure 8 – *Code 2 sur la carte : allumage par le bouton vert, extinction par le bouton rouge*

Après un appui bref sur le bouton vert, la LED reste allumée ; un appui sur le bouton rouge l'éteint, et elle reste éteinte jusqu'au prochain appui sur le vert.

6.4 Débogage et registres

Trois situations ont été observées : aucun bouton appuyé, appui sur le bouton vert (IDR = 0x2) puis appui sur le bouton rouge (IDR = 0x4).

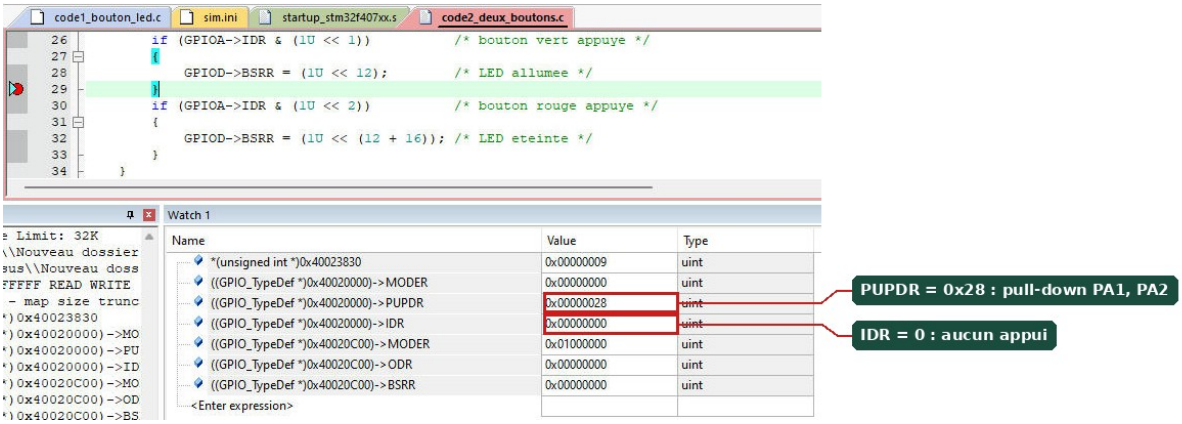


Figure 9 – *Code 2 au repos : pull-down sur PA1 et PA2, aucun bouton appuyé*

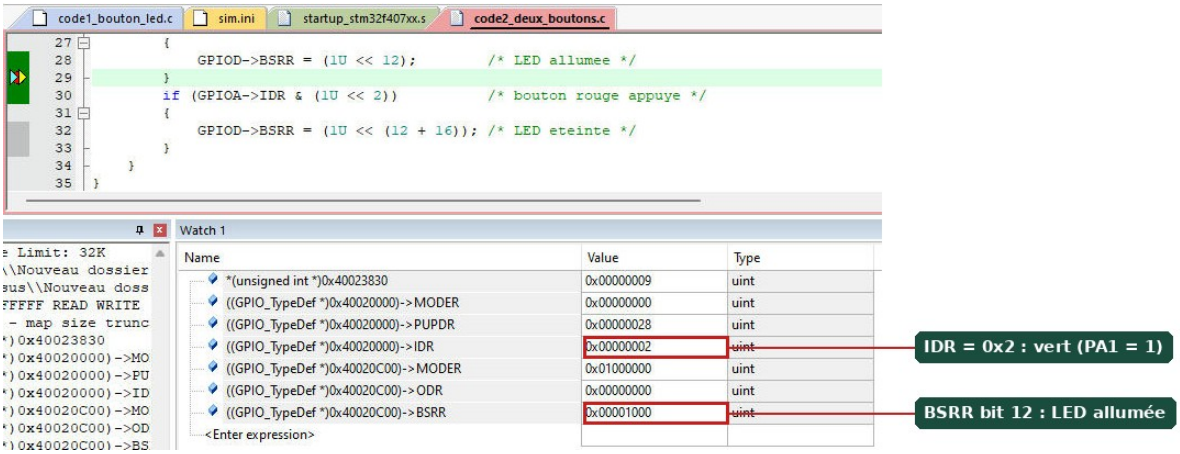


Figure 10 – *Code 2, appui sur le bouton vert : IDR = 0x2, BSRR = 0x1000*

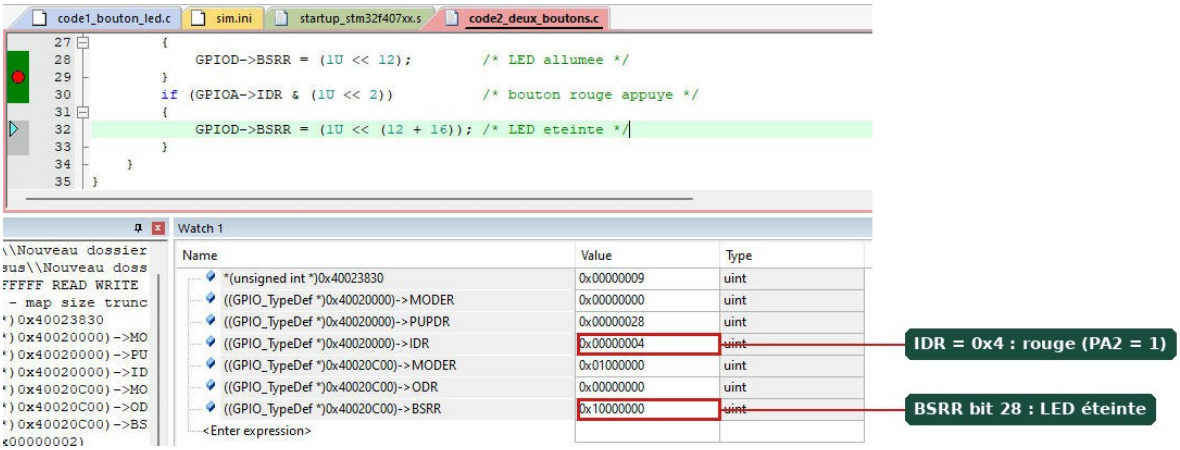


Figure 11 – *Code 2, appui sur le bouton rouge : IDR = 0x4, BSRR = 0x10000000*

Registre	Repos	Vert appuyé	Rouge appuyé	Interprétation
`GPIOA->PUPDR`	0x28	0x28	0x28	Pull-down (10) sur PA1 et PA2.
`GPIOA->IDR`	0x0	0x2	0x4	Bit 1 : bouton vert ; bit 2 : bouton rouge.
`GPIOD->BSRR`	0x0	0x1000	0x10000000	Rien au repos ; bit 12 : allumage ; bit 28 : extinction.

Au repos, BSRR reste à 0 : le programme n'écrit rien, ce qui confirme que la LED garde son état précédent. Chaque bouton provoque l'écriture du bit attendu.

7. Remarque : BSRR et ODR en simulation

Dans les captures, `GPIOD->ODR` reste à 0 même lorsque le programme demande d'allumer la LED. Ce n'est pas une erreur du programme : le simulateur de Keil, utilisé sans modèle du périphérique GPIO, traite les registres comme de la simple mémoire. L'écriture dans BSRR est bien enregistrée, mais l'action matérielle qui en découle, la mise à jour de ODR, n'est pas reproduite.

Sur la carte réelle, BSRR est un registre en écriture seule : écrire `0x00001000` met immédiatement le bit 12 de ODR à 1 (LED allumée), et écrire `0x10000000` le remet à 0. C'est ce que montrent les photos des parties 5.3 et 6.3. Pour observer ODR directement, il suffit de déboguer avec le ST-LINK, carte branchée, au lieu du simulateur.

8. Conclusion

Ce TP nous a permis de lire des entrées numériques du STM32F407 en programmant directement ses registres. Nous avons réalisé le montage d'un bouton poussoir avec une résistance de rappel vers la masse, configuré les entrées (MODER, PUPDR), lu leur état dans IDR et commandé une LED avec BSRR.

Le code 1 fait suivre à la LED l'état d'un bouton, tandis que le code 2 réalise une commande marche/arrêt avec deux boutons, l'arrêt étant prioritaire. Le débogueur de Keil nous a permis de vérifier chaque registre et de comprendre la différence entre BSRR et ODR. Une suite naturelle serait de remplacer la scrutation permanente des boutons par des interruptions externes (EXTI) et de traiter les rebonds mécaniques des boutons.

Remerciements

Nous remercions M. Atef Ftirich, notre enseignant de TP, pour ses explications claires, ses conseils et sa disponibilité tout au long de cette séance. Nous remercions également l'établissement pour la mise à disposition du matériel nécessaire à la réalisation de ce travail.